Test Modern mode with Playwright

You are a new joiner. This guide tells you how to add the test that a Modern change needs. It also tells you what a module must cover before its Modern migration counts as finished.

Read the whole page once. After that, use the section for the task in front of you.

Follow three rules first

The rules come from `AGENTS.md`. A test that breaks one of them will be sent back, however green it is.

1. Every Modern behaviour sits behind the Modern gate. If your change shows in Classic mode, it is wrong.
2. A Modern change never alters Classic behaviour. Test the Modern path, and check that the Classic path still works.
3. Read the interface mode only through `Mode`. In a view, use `Mode.IsModern`, `Mode.IsClassic`, `Mode.ModernOnly` or `Mode.IsModernJs`. Outside a view, use `Mode.For(HttpContext)`. Do not test or write `ModernShellPreference`.

Prefer the Modern JavaScript and CSS layers. Treat a `.cshtml` change as a last resort. Adding or removing a data attribute is allowed. Any other `.cshtml` change must be gated to Modern mode.

Set up once

Install the packages and the browser.

```
npm run visual:install
```

Run the component tier. It needs nothing else, and it is the tier that gatekeeps a pull request.

```
npm run test:visual:component
```

Run the live tier when your test opens a real page. Start the host in a second terminal first.

```
npm run dev:modern
npm run test:visual:live
```

The live tier reads `WALLETHR_BASE_URL` and the test credentials. The defaults point at the local host and the local seed, so you do not set anything to run it. The full list is in Environment variables.

Pick the kind of test

Three kinds exist. Pick one before you write a line.

Kind	Use it when	It needs	It gates CI
Component contract	The change is in a Modern adapter, a shell layer, a view's markup, or a data attribute	The repository only	Yes
Live page	The change needs the real page, the session, a role, or server data	The host and the seed	No
Module parity	You are proving that a whole module behaves the same in Modern as in Classic	The host and the seed	No

Most Modern work needs a component contract. It runs in about 20 milliseconds and it cannot fail because a database row moved.

Write a component contract

The spec supplies the markup that the page supplies. It loads the real styles and the real adapters from `wwwroot`. It asserts what a person would see.

1. Name the file `tests/visual/modern-<surface>.spec.js`, where the surface is the thing you changed.
2. Copy the markup from the view that renders it. Do not invent markup that the page never produces, or the test proves nothing.
3. Load the real assets. Use `loadModernStyles(page)` from `tests/visual/modern-styles.js` for the styles. Use `pathFor(name)` or `loadAdapterPaths(page, paths)` from `tests/visual/modern-component-adapters.js` for the adapters.
4. Drive the control as a person would. Click the button. Type in the field. Press `Tab` or `Escape` where the keyboard matters.
5. Assert the outcome that a person sees. The visible text, the value that the form will post, the element that has focus, the request that left the page.

This is the shape of a component contract.

```
const path = require('path');
const { test, expect } = require('@playwright/test');
const { pathFor } = require('./modern-component-adapters');
const { loadModernStyles } = require('./modern-styles');

const root = path.resolve(__dirname, '../..');
const origin = 'https://wallethr.test';

test('replaces the icon with the loader while a save runs', async ({ page }) => {
  await page.route(`${origin}/**`, route => route.fulfill({ json: { data: { ok: true } } }));
  await page.setContent(`<body data-wh-modern-shell><main id="wh-modern-page">
    <button id="save" class="btn" data-variant="primary" type="button">Save</button></main></body>
  await page.goto(`${origin}/`);
  await loadModernStyles(page);
  await page.addScriptTag({ path: path.join(root, 'Crystal.Core.Web/wwwroot/Content/modern/core/ru
  await page.addScriptTag({ path: pathFor('button') });

  await page.locator('#save').click();
  await expect(page.locator('#save')).toHaveAttribute('aria-busy', 'true');
});
```

`tests/visual/modern-button-busy.spec.js` is the shortest working example. Copy its structure.

Rules for a component contract

- Do not read `Crystal.Core.Web/wwwroot/Content/modern/dist`. The build writes that folder and git ignores it. Load the source file that the build uses.
- Do not assert a CSS value that only a designer cares about. Assert the state the value expresses, such as `aria-expanded` or `toBeVisible()`.
- Do not use a fixed sleep. Wait for the thing itself with a Playwright expectation, or a `page.route` handler that you release when you choose.
- Fix the clock with `page.clock.setFixedTime()` when the result depends on today. A date assertion that passes in April and fails in May is a bug in the test.
- Keep one scenario in one test. A test named after two behaviours fails in a way that names neither.
- If the behaviour needs a fixture with many rows, build the fixture in the spec. Do not depend on the local database.

Write a live page test

Use a live page test when the subject is the page itself. Examples are a role that hides a menu, a saved record that renders, or a redirect after sign in.

1. Put the file in `tests/visual` and let the runner see the credential variables, so it puts the file in the live tier automatically.
2. Sign in with the shared helper. Copy `login(page)` from `tests/visual/interface-mode-isolation.spec.js`.
3. Keep the test out of the pull request gate. It runs on your machine, before you push, and in the nightly run when one exists.
4. Leave the database as you found it. Do not delete, terminate, or process a real record to make a test pass.

Write a module scenario test

A module scenario test drives the real module from the real menu. It is the test that answers "does this module work", and it is what a client asks for.

The repository already runs this pattern for Core HR end to end. The runner takes a module name, so the same pattern works for every module. These are the parts.

- `tests/visual/module-regression/modules/<module>.js` defines the module. It holds the name, the authorization profile, the viewports, and the list of methods and actions that the run must not send.
- `specs/<module>/README.md` is the written register for the module. It lists the scenarios and the bug tests, and it says which ones pass.
- `npm run ui:test-module -- --module="<Module name>"` discovers the module's routes and compares the Classic and Modern versions of each one.
- The phases are `discover`, `verify`, `behavior`, `modern-behavior`, and `recompare`. Use `discover` first, because later phases read its report.
- The report lands in `.run/module-regression/<module>/latest/`. Read `summary.md` first.

Start a new module with these steps.

1. Copy `tests/visual/module-regression/modules/corehr.js` and change the name, the profile, and the viewports.
2. Run the discovery phase and read the route inventory.
3. Write `specs/<module>/README.md` with the scenario register from the matrix below.
4. Add the behaviour phases one route at a time. Fix the locators, then the waits, then the setup. Never remove an assertion to make a run green.
5. Keep the safety list tight. A scenario that saves, deletes, approves, or pays must run on a record that the run created and can recognise by name.

What a module must cover

Every module is expected to carry these scenario classes for its main screens. The matrix is the standard. A module is finished when every row has a test, or a line in `specs/<module>/README.md` that says why the row does not apply.

Scenario class	The check that must exist
List and search	The list renders the seeded rows. Search narrows it. The empty result says so.
Filters and paging	A filter changes the rows. A page change keeps the filter. Clearing the filter restores the rows.
Create and edit	Every required field blocks the save with its message. A valid save reaches the server once. Cancel leaves the record unchanged.
Validation	Each message appears next to its field. A message clears when the field becomes valid.
Delete and confirm	The confirm dialog names the record. Cancel changes nothing. The record leaves the list after the confirm.
Detail and tabs	Each tab loads its own content. A tab that fails shows an error, not an empty panel.

Scenario class	The check that must exist
Permissions	A role with the right sees the action. A role without it does not see the action, and the direct URL does not serve it either.
Mode parity	The same task reaches the same outcome in Classic and in Modern. Turning the Modern gate off gives the Classic screen, unchanged.
Viewports	The screen works at 1440, at 1024, and at 390 pixels wide. Nothing overlaps and nothing scrolls sideways.
Data edges	A long name, a name with a non-Latin script, zero rows, and the maximum row count all render without breaking the layout.
Exports and reports	The file arrives in the right format with the filtered rows. An empty result gives a clear message, not an empty file.
Failure paths	A failed request shows an error the person can read, and leaves the page usable.

Two rules apply to every row.

- Test the Modern path. Then check that the Classic path is unchanged, by running the same spec with the gate off or by using the parity runner.
- Put the assertion on what the person can observe. A test that reaches into a JavaScript object proves that the object changed, not that the module works.

Cover every bug fix

A fixed bug gets a test in the same pull request as the fix. No exceptions, including a one-line copy change.

1. Write the assertion against the broken state first. Watch it fail. If it passes before your fix, it is not testing the bug.
2. Name the test for the report, in the words the person who reported it used. “Keeps the Policy filter after the log closes” beats “fix dialog”.
3. Put the test next to the surface it protects, in the spec file for that surface. Do not make a file that holds only old bugs.
4. Keep it after the fix, for good. A deleted regression test buys nothing.
5. Add the row to `specs/<module>/README.md`, with the report name and the date.

Run and debug

Run the tier that your change belongs to. Do not run the whole directory.

```
npm run test:visual:component                    # the gate
node scripts/run-visual-tier.mjs --tier=component --file=modern-button-busy   # one file
npm run test:visual:tiers                        # which specs are in which tier
```

When a test fails, read these in order.

1. The error and the locator it names. Most failures are a locator that matched two elements, or a name that changed.
2. `.run/playwright-output/<test-name>/test-failed-1.png`. That is the screen at the moment of failure.
3. `.run/playwright-output/<test-name>/trace.zip` with `npx playwright show-trace`. Use it for anything that involves timing.
4. The spec’s own assumption. A test that broke after a rename is a stale test. A test that broke after a behaviour change is a bug report.

Do not update a snapshot or a baseline to make a failure go away until you can say what changed on the screen and why that is correct.

If a test fails only sometimes, the test is wrong or the code has a race. The fix is never a retry. Recent examples were a timer in the combobox adapter and a duplicate Apply button on a page with two providers.

Ship it

Before you open the pull request, confirm each line.

- The component tier passes, and so does any live test that covers the same surface.
- The change shows in Modern mode and in no other mode.
- Classic mode behaves exactly as it did before.
- Every new behaviour has an assertion on something a person can see.
- Every fixed bug has a test that failed before the fix.
- The module register in `specs/<module>/README.md` lists anything new.
- No test asserts a value from `modern/dist`, and no test sleeps.

Reference

Test families that exist

The suite holds 113 spec files and 764 tests. 62 files are in the component tier and 51 are in the live tier. A file is named after the surface it covers, so find the nearest neighbour by name.

Family	What it covers
`modern-home-*`, `modern-dashboard-*`, `modern-analytics-*`	Home cards, the dashboard, analytics cards and charts
`modern-form-*`, `modern-validation-*`, `modern-input-*`, `modern-variable-input-*`	Forms, fields, validation, the variable input control
`modern-append-options`, `modern-dropdown-data`, `modern-attribute-filter-*`	Data-bound controls and their providers
`modern-business-provider`, `modern-calendar-provider`, `modern-table-provider`, `modern-tabs-provider`	The adapter boundary for each component family
`modern-select2-*`, `modern-date-picker-*`, `modern-combobox-*`, `modern-tree-*`	The Classic plugin replacements
`modern-shell-*`, `interface-mode-isolation`, `route-mode-isolation`	The shell, navigation, progress, and mode isolation
`modern-exit-*`, `modern-event-*`, `modern-joining-*`, `modern-associate-*`, `modern-recruitment-*`, `modern-policy-*`	Module screens that were migrated
`modern-superadmin-*`, `modern-administrator-*`, `manage-*`	Administrator and user management screens
`classic-*`, `wallethr-visual`	The Classic screens that must not change
`module-regression/*`	The Classic and Modern parity harness for a whole module

Three specs are evidence captures rather than gates. They record a before and after video of a review, so they skip unless you set their address.

Environment variables

The live tier and the parity runner read these. The defaults are the local seed, so a local run needs no setup.

Variable	Default	Used for
`WALLETHR_BASE_URL`	`http://127.0.0.1:7205`	The host that the live tier and the parity runner open
`WALLETHR_TEST_COMPANY_ID`	`1`	The company that the login helper selects
`WALLETHR_TEST_USERNAME`	`1003`	The user that the login helper signs in
`WALLETHR_TEST_PASSWORD`	`1`	The password for that user
`WALLETHR_TEST_SUPERADMIN_USERNAME`	none	The user for the Administrator screens
`WALLETHR_TEST_SUPERADMIN_PASSWORD`	none	The password for that user
`WALLETHR_RUN_LIVE_SUPERADMIN`	off	Set 1 to run the Administrator checks that need that user
`WALLETHR_AUTOFILL_TEST_VALUES`	off	Set `true` to let the parity runner fill form values
`WALLETHR_MODULE_NAME`	`Core HR`	The module that the parity runner opens
`WALLETHR_ALLOW_LOCAL_WRITES`	off	Set `true` to let the parity runner send a write to the local host
`WALLETHR_STORAGE_STATE`	none	A saved sign-in for the parity runner, when the seed cannot sign in on its own
`WALLETHR_UI_MODE`	`modern`	The mode that `test:visual:bulk` captures. `classic` or `modern`
`WALLETHR_VIEWPORT`	`desktop`	The viewport that `test:visual:bulk` captures. `desktop` or `mobile`
`PLAYWRIGHT_BROWSERS`	`chromium`	The browsers that `visual:install` downloads
`PLAYWRIGHT_WITH_DEPS`	off	Set 1 on a Linux runner to install the system libraries
`WALLETHR_GALLERY_URL`, `WALLETHR_CLOSURE_BASE_URL`, `WALLETHR_EXPORT_LOCAL`	none	The addresses for the three evidence captures. They skip until you set one

Scripts

Script	What it does
`visual:install`	Installs the packages and the Chromium browser, then checks the result
`test:visual:component`	Runs the component tier against the recorded baseline. This is the gate
`test:visual:live`	Runs the live tier. It stops if the host does not answer
`test:visual:tiers`	Lists which specs are in which tier
`test:visual:baseline`	Records the current component failures as known failures
`test:visual`	Runs the whole directory, serially
`test:visual:update`	Updates the screenshot snapshots
`ui:test-module`	Runs the module parity runner for one module
`ui:test-corehr:*`	Runs one phase of the Core HR parity work
`audit:classic-modern`	Audits every module route in both modes and builds a dashboard
`test:visual:all-routes`	Audits every module route in Modern mode

The component tier is configured to run in CI on the `windows-om-build` runner, through `.gitea/workflows/visual.yml`. That workflow has not had its first run yet. Until it does, treat the component tier as a local gate. The live tier stays local in any case, because the runner has no seeded database.